



Relatório do Projeto de Estruturas de Dados Avançados

Tiago Rodrigo Soares Afonso

Nº 20452 – Regime Diurno

Professor

Luís Gonzaga Martins Ferreira

Ano letivo 2024/2025

Licenciatura em Engenharia de Sistemas Informáticos

Escola Superior de Tecnologia

Instituto Politécnico do Cávado e do Ave

ÍNDICE

1. Introdução	VI
1.1. Objetivos.....	VII
1.2. Fase 1 – Listas Ligadas	VII
1.3. Fase 2 – Grafos	VII
2. Fase 1 – Listas Ligadas	IX
2.1. Definição da Estrutura de Dados	IX
2.2. Antena	XI
2.2.1. Cabeçalho (Antena.h)	XI
2.2.2. Implementação (Antena.cpp)	XII
2.3. Lista Antenas	XV
2.3.1. Cabeçalho (ListaAntenas.h)	XV
2.3.2. Implementação (ListaAntenas.cpp)	XV
2.4. Local Nefasto.....	XIX
2.4.1. Cabeçalho (LocalNefasto.h).....	XIX
2.4.2. Implementação (LocalNefasto.cpp):.....	XIX
2.5. Projeto_EDA.cpp (main.c)	XXIII
2.5.1. exibirMenu (Menu Interativo).....	XXIII
2.5.2. main (Função principal).....	XXIII
2.6. Testes ao programa.....	XXV
2.6.1. Menu	XXV
2.6.2. Listar Antenas	XXV
2.6.3. Listar Antenas Formatado	XXV
2.6.4. Inserir Antena.....	XXVI
2.6.5. Remover Antena	XXVI
2.6.6. Listar Locais Nefastos	XXVII

2.6.7.	Listar Locais Nefastos Formatado	XXVII
3.	Fase 2 – Grafos.....	XXVIII
3.1.	Atualização da estrutura de dados	XXVIII
3.2.	Grafo	XXIX
3.2.1.	Cabeçalho (Grafo.h)	XXIX
3.2.2.	Implementação (Grafo.cpp)	XXX
3.3.	Projeto_EDA.cpp atualizado.....	XXXVII
3.3.1.	Menu atualizado (exibirMenu).....	XXXVIII
3.3.2.	Main (função principal)	XXXVIII
3.4.	Novos testes ao programa	XXXIX
3.4.1.	Listagem de Vértices	XXXIX
3.4.2.	Listagem de Arestas	XXXIX
3.4.3.	Procura por Profundidade.....	XL
4.	Conclusão	XLI
5.	Bibliografia.....	XLII

ÍNDICE DE FIGURAS

Figura 1 - Estruturas de Dados	X
Figura 2 - Antena.h.....	XI
Figura 3 - Verifica posição Antena	XII
Figura 4 - Insere Antena.....	XIII
Figura 5 - Remover Antena	XIII
Figura 6 - Inserir e remover Antenas Manual	XIV
Figura 7 - Lista de Antenas (h).....	XV
Figura 8 - Criação da Estrutura de Dados das Antenas	XV
Figura 9 - Destrói a Estrutura de Dados.....	XVI
Figura 10 - Carrega Antenas do Ficheiro	XVI
Figura 11 - Lista Antenas	XVII
Figura 12 - Lista Antenas de Forma Formatada.....	XVIII
Figura 13 - Cabeçalho LocalNefasto(.h).....	XIX
Figura 14 - Local Nefasto Existe	XIX
Figura 15 - Adiciona Local Nefasto	XX
Figura 16 - Calcula Locais Nefastos.....	XXI
Figura 17 - Listar Locais Nefastos.....	XXII
Figura 18 - Lista Locais Nefastos Formatado.....	XXII
Figura 19 - Menu Interativo	XXIII
Figura 20 - Função principal (Main).....	XXIV
Figura 21 - Menu	XXV
Figura 22 - Lista de Antenas Consola	XXV
Figura 23 - Lista Antenas Formatado Consola.....	XXV
Figura 24 - Inserir Antena Consola.....	XXVI
Figura 25 - Remover Antena Consola.....	XXVI
Figura 26 - Lista Locais Nefastos Consola.....	XXVII
Figura 27 - Lista Locais Nefastos Formatada Consola.....	XXVII
Figura 28 - Estrutura de Dados da Fase 2	XXIX
Figura 29 - Grafo.h	XXX
Figura 30 – criar um Grafo	XXXI
Figura 31 - Destruir Grafo	XXXI
Figura 32 - Carrega Grafo	XXXII

Figura 33 - Inserir Vértice	XXXIII
Figura 34 - Insere Aresta	XXXIV
Figura 35 - Listar Vértices.....	XXXV
Figura 36 - Listar Arestas.....	XXXVI
Figura 37 - Função auxiliar para procura de Profundidade.....	XXXVI
Figura 38 - Função que procura em Profundidade	XXXVII
Figura 39 - Menu atualizado	XXXVIII
Figura 40 - Novas funcionalidade do programa	XXXVIII
Figura 41 - Listar Vertices.....	XXXIX
Figura 42 - Listagem de Arestas.....	XXXIX
Figura 43 - Procura por Profundidade	XL

1. Introdução

Este projeto destina-se à avaliação dos conhecimentos adquiridos na unidade curricular de Estruturas de Dados Avançadas (EDA) e foca-se na implementação e manipulação de estruturas de dados dinâmicas, bem como na aplicação de conceitos avançados de teoria dos grafos. O seu desenvolvimento, realizado em linguagem C, propõe a resolução de um problema de dimensão média através da definição de estruturas de dados específicas para a representação de antenas e respetivos efeitos nefastos, decorrentes de interações baseadas em frequências de ressonância.

A primeira fase do projeto centra-se na utilização de listas ligadas para a gestão de dados referentes às antenas e às localizações afetadas, abrangendo operações de inserção, remoção e dedução automática dos pontos críticos. Na segunda fase, o desafio é reinterpretado através de uma abordagem baseada em grafos, onde cada vértice representa uma antena e as arestas ligam antenas com frequências iguais, permitindo a exploração do espaço através de buscas em profundidade e largura, e a identificação de caminhos e intersecções entre pares de antenas de frequências distintas.

Esta iniciativa não só reforça a compreensão dos conceitos teóricos, mas também promove a sua aplicação prática através de um desenvolvimento modular e rigorosamente documentado.

O projeto constitui, assim, uma oportunidade de integrar e aplicar conhecimentos de estruturas de dados avançadas, contribuindo para a formação de competências essenciais à resolução de problemas computacionais complexos.

1.1. Objetivos

- Consolidar os conhecimentos adquiridos na unidade curricular de Estruturas de Dados Avançadas (EDA) através da aplicação prática em linguagem C.
- Implementar estruturas de dados dinâmicas para a representação de antenas, utilizando listas ligadas na Fase 1 e grafos na Fase 2.
- Desenvolver um mecanismo para carregar e interpretar os dados de antenas a partir de um ficheiro de texto, onde as antenas são representadas numa matriz de caracteres de dimensões variáveis.

1.2. Fase 1 – Listas Ligadas

- ✓ Definir e manipular uma estrutura de dados (lista ligada) para armazenar a frequência de ressonância e as coordenadas das antenas.
- ✓ Implementar operações de inserção e remoção de antenas na lista ligada.
- ✓ Deduzir automaticamente as localizações com efeito nefasto, conforme a regra de alinhamento e distância entre pares de antenas com a mesma frequência, e representá-las através de uma lista ligada.
- ✓ Exibir, de forma tabular na consola, tanto os dados das antenas como as localizações identificadas com efeito nefasto.

1.3. Fase 2 – Grafos

- Definir um tipo de dados para a representação de grafos (GR), onde cada vértice representa uma antena e as arestas conectam antenas com frequências de ressonância iguais.
- Carregar os dados das antenas de um ficheiro de texto para construir o grafo correspondente.
- Implementar operações de manipulação de grafos, que incluam:
 - Procura em profundidade (DFS) a partir de uma antena, listando as coordenadas das antenas alcançadas.
 - Procura em largura (BFS) a partir de uma antena, com listagem semelhante.

- Identificação e listagem de todos os caminhos possíveis entre duas antenas, apresentando as sequências de arestas interligadas.
- Listagem de intersecções de pares de antenas com frequências distintas, indicando as respectivas coordenadas.

2. Fase 1 – Listas Ligadas

A Fase 1 do projeto centrou-se na implementação de estruturas de dados dinâmicas através da utilização de listas ligadas para representar as antenas numa cidade. Cada antena foi identificada pela sua frequência de ressonância e coordenadas espaciais, armazenadas num registo da lista ligada. O objetivo foi desenvolver operações essenciais como inserção, remoção e dedução automática das localizações com efeito nefasto, com base na disposição e frequência das antenas. Além disso, será realizada uma listagem tabular na consola para visualizar tanto as antenas registadas como as localizações críticas identificadas.

2.1. Definição da Estrutura de Dados

Foram definidas estruturas de dados para representar antenas e locais nefastos utilizando listas ligadas em C no ficheiro **Dados.h**.

Este ficheiro tem os seguintes dados:

- **Antena:** Estrutura com a frequência (caracter), coordenadas (linha e coluna) e ponteiro para a próxima antena.
- **ED:** Estrutura principal que contém o ponteiro para a cabeça da lista de antenas.
- **LocalNefasto:** Estrutura com coordenadas (linha e coluna) e ponteiro para o próximo local nefasto.
- **ED_LocaisNefastos:** Estrutura principal que contém o ponteiro para a cabeça da lista de locais nefastos.

Pode observar-se a estrutura na **Figura 1**.

```

#ifndef DADOS_H
#define DADOS_H

/**
 * @brief Estrutura que representa uma antena na lista ligada
 */
typedef struct Antena {
    char frequencia;    ///< Frequência da antena (ex: 'A', 'O')
    int linha;          ///< Linha na matriz (coordenada x)
    int coluna;         ///< Coluna na matriz (coordenada y)
    struct Antena* prox; ///< Ponteiro para a próxima antena na lista
} Antena;

/**
 * @brief Estrutura principal ED (lista ligada de antenas)
 */
typedef struct {
    Antena* cabeca;    ///< Ponteiro para a primeira antena da lista
} ED;

/**
 * @brief Estrutura que representa um local nefasto na lista ligada
 */
typedef struct LocalNefasto {
    int linha;
    int coluna;
    struct LocalNefasto* prox;
} LocalNefasto;

/**
 * @brief Estrutura principal LocaisNefastos (lista ligada de locais nefastos)
 */
typedef struct {
    LocalNefasto* cabeca; // Lista de locais nefastos
} ED_LocaisNefastos;

#endif // DADOS_H

```

Figura 1 - Estruturas de Dados

2.2. Antena

2.2.1. Cabeçalho (Antena.h)

Nesta classe foram declaradas as funções para verificar a posição da antena, inserir e remover antenas, e manipular manualmente os dados das antenas.

Aqui os dados só estão declarados e não implementados.

```
#pragma once
#ifndef ANTENA_H
#define ANTENA_H

#include "Dados.h" // Para usar as estruturas Antena e ED

// ----- Operações Básicas -----
int verificaPosicaoAntena(ED* ed, int linha, int coluna);
void insereAntena(ED* ed, char freq, int linha, int coluna);
int removerAntena(ED* ed, char freq, int linha, int coluna);
void inserirAntenaManual(ED* ed);
void removerAntenaManual(ED* ed);

#endif // ANTENA_H
```

Figura 2 - Antena.h

Pode observar-se a declaração dos métodos sem a implementação na **Figura**

2.

2.2.2. Implementação (Antena.cpp)

Nesta classe foram implementados os métodos declarados anteriormente como:

verificaPosicaoAntena: Confirma se já existe uma antena numa posição específica percorrendo a lista ligada e verificando se alguma delas já está na posição específica (linha, coluna). **(Figura 3).**

```
int verificaPosicaoAntena(ED* ed, int linha, int coluna) {  
    // Ponteiro auxiliar para percorrer a lista de antenas  
    Antena* atual = ed->cabeca;  
  
    // Percorre a lista ligada até encontrar uma correspondência ou chegar ao final  
    while (atual != NULL) {  
        // Verifica se a antena atual está na mesma posição especificada  
        if (atual->linha == linha && atual->coluna == coluna) {  
            return 0; // Retorna 0 indicando que a posição já está ocupada  
        }  
        atual = atual->prox; // Avança para a próxima antena na lista  
    }  
  
    return 1; // Retorna 1 indicando que a posição está disponível  
}
```

Figura 3 - Verifica posição Antena

insereAntena: Insere uma nova antena no início da lista ligada, após verificar a disponibilidade da posição. Este método tem como argumentos de entrada a Lista Ligada, a frequência, a linha e a coluna da nova antena. **(Figura 4).**

```

void insereAntena(ED* ed, char freq, int linha, int coluna) {

    // Verifica se a posição já está ocupada por outra antena
    if (!verificaPosicaoAntena(ed, linha, coluna)) {
        printf("Erro: Ja existe uma antena em (%d, %d).\n", linha, coluna);
        return; // Sai da função sem inserir a nova antena
    }

    // Aloca memória para a nova antena
    Antena* novaAntena = (Antena*)malloc(sizeof(Antena));
    if (novaAntena == NULL) {
        perror("Erro ao alocar memoria para Antena");
        exit(EXIT_FAILURE); // Termina o programa em caso de erro de alocação
    }

    // Inicializa os valores da nova antena
    novaAntena->frequencia = freq;
    novaAntena->linha = linha;
    novaAntena->coluna = coluna;

    // Insere a nova antena no início da lista ligada
    novaAntena->prox = ed->cabeca;
    ed->cabeca = novaAntena;
}

```

Figura 4 - Insere Antena

removerAntena: Remove a antena que corresponde à frequência e coordenadas fornecidas caso ela realmente esteja presente na lista ligada, atualizando a lista ligada seguidamente. **(Figura 5)**

```

int removerAntena(ED* ed, char freq, int linha, int coluna) {

    Antena* atual = ed->cabeca; // Ponteiro para percorrer a lista
    Antena* anterior = NULL;    // Ponteiro para manter referência ao nó anterior

    // Percorre a lista até encontrar a antena ou chegar ao final
    while (atual != NULL) {

        // Verifica se a antena atual corresponde aos critérios de remoção
        if (atual->linha == linha && atual->coluna == coluna && atual->frequencia == freq) {

            // Se a antena a ser removida for a primeira da lista
            if (anterior == NULL) {
                ed->cabeca = atual->prox; // Atualiza a cabeça da lista
            }
            else {
                anterior->prox = atual->prox; // Salta o nó a ser removido
            }

            free(atual); // Liberta a memória da antena removida
            return 1; // Retorna 1 indicando remoção bem-sucedida
        }

        // Avança para o próximo nó, mantendo o anterior registrado
        anterior = atual;
        atual = atual->prox;
    }

    return 0; // Retorna 0 caso a antena não tenha sido encontrada na lista
}

```

Figura 5 - Remover Antena

inserirAntenaManual/removerAntenaManual: Permite ao utilizador introduzir os dados da antena que deseja inserir ou remover pela consola. Assim estas funções guardam os argumentos inseridos e são utilizados nos métodos de inserir e remover criados anteriormente. (Figura 6)

```
// Função para inserir uma antena manualmente
void inserirAntenaManual(ED* ed) {
    char freq;
    int linha, coluna;

    // Ler frequência
    printf("Insira a frequência da antena (caractere) a acrescentar: ");
    scanf_s(" %c", &freq, 1);

    // Ler coordenadas
    printf("Insira a linha da antena: ");
    scanf_s("%d", &linha);

    printf("Insira a coluna da antena: ");
    scanf_s("%d", &coluna);

    // Chamar função de inserção
    insereAntena(ed, freq, linha, coluna);
}

// Função para remover uma antena manualmente
void removerAntenaManual(ED* ed) {
    char freq;
    int linha, coluna;

    // Ler frequência
    printf("Insira a frequência da antena (caractere) a remover: ");
    scanf_s(" %c", &freq, 1);

    // Ler coordenadas
    printf("Insira a linha da antena: ");
    scanf_s("%d", &linha);

    printf("Insira a coluna da antena: ");
    scanf_s("%d", &coluna);

    // Chamar função de remoção
    removerAntena(ed, freq, linha, coluna);
}
```

Figura 6 - Inserir e remover Antenas Manual

2.3. Lista Antenas

2.3.1. Cabeçalho (ListaAntenas.h)

Neste ficheiro foram declaradas funções para criar e destruir a estrutura de dados (ED), carregar antenas a partir de ficheiros de texto, e listar as antenas de diferentes formas. **(Figura 7)**

```
#pragma once

#ifndef LISTAANTENAS_H
#define LISTAANTENSA_H

#include "Dados.h" // Para usar as estruturas Antena e ED

// ----- Gestão da ED -----
ED* criarED();
void destruirED(ED* ed);
void carregarAntenasDeFicheiro(ED* ed, const char* nomeFicheiro, int* max_linhas, int* max_colunas);
void listarAntenas(const ED* ed);
void listarAntenasFormatado(const ED* ed, int max_linhas, int max_colunas);

#endif // ANTENA_H
```

Figura 7 - Lista de Antenas (h)

2.3.2. Implementação (ListaAntenas.cpp)

Nesta classe foram implementados os métodos declarados anteriormente como:

criarED: Aloca memória para a estrutura e inicializa a lista como vazia. **(Figura 8)**

```
ED* criarED() {
    // Aloca memória para a estrutura ED
    ED* ed = (ED*)malloc(sizeof(ED));
    if (ed == NULL) {
        perror("Erro ao alocar memória para ED");
        exit(EXIT_FAILURE); // Encerra o programa em caso de erro na alocação
    }

    ed->cabeca = NULL; // Inicializa a lista como vazia
    return ed; // Retorna o ponteiro para a estrutura criada
}
```

Figura 8 - Criação da Estrutura de Dados das Antenas

destruirED: Liberta toda a memória alocada para a lista ligada, removendo cada antena. **(Figura9).**

```

void destruirED(ED* ed) {
    Antena* atual = ed->cabeca; // Ponteiro para percorrer a lista

    // Percorre a lista, liberando a memória de cada antena
    while (atual != NULL) {
        Antena* prox = atual->prox; // Guarda referência para o próximo nó
        free(atual); // Liberta a memória da antena atual
        atual = prox; // Avança para o próximo nó
    }

    free(ed); // Liberta a memória da estrutura ED
}

```

Figura 9 - Destrói a Estrutura de Dados

carregarAntenasDeFicheiro: Lê um ficheiro de texto contendo uma matriz de caracteres. Cada célula pode conter uma antena ou um caractere "." indicando espaço vazio. Insere antenas na lista conforme encontrado no ficheiro. **(Figura 10)**

```

void carregarAntenasDeFicheiro(ED* ed, const char* nomeFicheiro, int* max_linhas, int* max_colunas) {
    FILE* arquivo = NULL;

    // Abre o ficheiro para leitura
    errno_t err = fopen_s(&arquivo, nomeFicheiro, "r");
    if (err != 0 || arquivo == NULL) {
        perror("Erro ao abrir o arquivo");
        return;
    }

    char buffer[1024]; // Buffer para armazenar cada linha lida do ficheiro
    int linhaAtual = 0;
    *max_colunas = 0;

    // Lê o ficheiro linha por linha
    while (fgets(buffer, sizeof(buffer), arquivo) != NULL) {
        size_t comprimento = strlen(buffer);

        // Remove o caractere de nova linha ('\n') se presente
        if (comprimento > 0 && buffer[comprimento - 1] == '\n') {
            buffer[comprimento - 1] = '\0';
            comprimento--;
        }

        // Atualiza o número máximo de colunas encontrado
        if (comprimento > *max_colunas) {
            *max_colunas = comprimento;
        }

        // Processa cada caractere da linha
        for (int coluna = 0; coluna < comprimento; coluna++) {
            if (buffer[coluna] != '.') { // Se não for '.', é uma antena
                insereAntena(ed, buffer[coluna], linhaAtual, coluna);
            }
        }

        linhaAtual++; // Avança para a próxima linha
    }

    *max_linhas = linhaAtual; // Atualiza o número total de linhas lidas
    fclose(arquivo); // Fecha o ficheiro após a leitura
}

```

Figura 10 - Carrega Antenas do Ficheiro

Listagem de antenas ordenadas:

Conta o número de antenas na lista. Armazena os ponteiros num array e ordena-os por linha (decrescente) e coluna (crescente). Exibe as antenas ordenadas na consola. (Figura 11)

```
void listarAntenas(const ED* ed) {
    // Passo 1: Contar o número total de antenas na lista
    int numAntenas = 0;
    Antena* atual = ed->cabeca;
    while (atual != NULL) {
        numAntenas++;
        atual = atual->prox;
    }

    // Passo 2: Alocar um array de ponteiros para armazenar as antenas
    Antena** arrayAntenas = (Antena**)malloc(numAntenas * sizeof(Antena*));
    if (arrayAntenas == NULL) {
        perror("Erro ao alocar memória");
        return;
    }

    // Passo 3: Preencher o array com as antenas da lista ligada
    atual = ed->cabeca;
    for (int i = 0; i < numAntenas; i++) {
        arrayAntenas[i] = atual;
        atual = atual->prox;
    }

    // Passo 4: Ordenar as antenas primeiro por linha (decrescente), depois por coluna (crescente)
    qsort(arrayAntenas, numAntenas, sizeof(Antena*),
    [](const void* a, const void* b) -> int {
        Antena* antenaA = *(Antena**)a;
        Antena* antenaB = *(Antena**)b;

        // Ordenação decrescente por linha
        if (antenaA->linha != antenaB->linha) {
            return antenaB->linha - antenaA->linha; // Inverter a ordem
        }

        // Se as linhas forem iguais, ordenar por coluna de forma crescente
        return antenaA->coluna - antenaB->coluna;
    });

    // Passo 5: Exibir a lista de antenas ordenadas
    printf("=== Lista de Antenas (Ordenadas) ===\n");
    for (int i = 0; i < numAntenas; i++) {
        printf("Antena '%c' em (%d, %d)\n",
            arrayAntenas[i]->frequencia,
            arrayAntenas[i]->linha,
            arrayAntenas[i]->coluna);
    }
    printf("=====\n");

    // Passo 6: Libertar a memória alocada para o array de ponteiros
    free(arrayAntenas);
}
```

Figura 11 - Lista Antenas

Listagem formatada de antenas (formato de Ficheiro de Entrada):

Cria uma matriz que representa a disposição das antenas. Preenche a matriz com caracteres “.” e substitui pelas frequências das antenas conforme a lista. Imprime a matriz de forma visualmente organizada. (Figura 12)

```
void listarAntenasFormatado(const ED* ed, int max_linhas, int max_colunas) {
    if (ed == NULL || ed->cabeca == NULL) {
        printf("Nenhuma antena encontrada.\n");
        return;
    }

    // Alocar dinamicamente uma matriz de caracteres (array de strings)
    char** matriz = (char**)malloc(max_linhas * sizeof(char*));
    if (matriz == NULL) {
        perror("Erro ao alocar memória para a matriz");
        exit(EXIT_FAILURE);
    }

    // Alocar e inicializar cada linha da matriz
    for (int i = 0; i < max_linhas; i++) {
        matriz[i] = (char*)malloc((max_colunas + 1) * sizeof(char)); // +1 para '\0'
        if (matriz[i] == NULL) {
            perror("Erro ao alocar memória para uma linha da matriz");
            exit(EXIT_FAILURE);
        }
        // Preencher a linha com '.' e adicionar o terminador de string '\0'
        for (int j = 0; j < max_colunas; j++) {
            matriz[i][j] = '.';
        }
        matriz[i][max_colunas] = '\0';
    }

    // Percorrer a lista de antenas e marcar suas posições na matriz
    Antena* atual = ed->cabeca;
    while (atual != NULL) {
        if (atual->linha >= 0 && atual->linha < max_linhas &&
            atual->coluna >= 0 && atual->coluna < max_colunas) {
            matriz[atual->linha][atual->coluna] = atual->frequencia; // Marca a antena na matriz
        }
        atual = atual->prox;
    }

    // Imprimir a matriz formatada na consola
    printf("=== Mapa de Antenas ===\n");
    for (int i = 0; i < max_linhas; i++) {
        printf("%s\n", matriz[i]); // Imprime cada linha da matriz
        free(matriz[i]); // Libera a memória da linha após a impressão
    }

    // Liberar memória alocada para o array de ponteiros
    free(matriz);
    printf("=====\n");
}
```

Figura 12 - Lista Antenas de Forma Formatada

2.4. Local Nefasto

2.4.1. Cabeçalho (LocalNefasto.h)

Declarou-se aqui as funções para calcular e listar locais nefastos. (Figura 13)

```
#pragma once
#ifndef LOCALNEFASTO_H
#define LOCALNEFASTO_H

#include "Dados.h" // Já define ED_LocaisNefastos

// Função principal
void calcularLocaisNefastos(ED* ed, ED_LocaisNefastos* locaisNefastos, int max_linhas, int max_colunas);

// Função auxiliar (não declarada no .h)
void listarLocaisNefastos(const ED_LocaisNefastos* locais);

void listarLocaisNefastosFormatado(const ED_LocaisNefastos* locais, int max_linhas, int max_colunas);

#endif // LOCALNEFASTO_H
```

Figura 13 - Cabeçalho LocalNefasto(.h)

2.4.2. Implementação (LocalNefasto.cpp):

Nesta classe foram implementados os métodos declarados anteriormente como:

localExiste: Confirma se um local nefasto já está presente na lista. (Figura 14).

```
// Verifica se um local nefasto já existe na lista
static int localExiste(ED_LocaisNefastos* locais, int linha, int coluna) {
    LocalNefasto* atual = locais->cabeca;
    while (atual != NULL) {
        if (atual->linha == linha && atual->coluna == coluna) return 1;
        atual = atual->prox;
    }
    return 0;
}
```

Figura 14 - Local Nefasto Existe

adicionaLocalNefasto: Adiciona um novo local nefasto à lista, evitando duplicações. (Figura 15)

```

// Adiciona um local nefasto à lista
static void adicionarLocalNefasto(ED_LocaisNefastos* locais, int linha, int coluna) {
    if (localExiste(locais, linha, coluna)) return;

    LocalNefasto* novo = (LocalNefasto*)malloc(sizeof(LocalNefasto));
    if (novo == NULL) {
        perror("Erro ao alocar memória para LocalNefasto");
        exit(EXIT_FAILURE);
    }

    novo->linha = linha;
    novo->coluna = coluna;
    novo->prox = locais->cabeca;
    locais->cabeca = novo;
}

```

Figura 15 - Adiciona Local Nefasto

calcularLocaisNefastos:

Agrupa antenas por frequência usando uma estrutura de dados temporária.

Para cada par de antenas da mesma frequência, calcula possíveis locais nefastos.

Garante que as posições calculadas estejam dentro dos limites da matriz.
(Figura 16)

```

void calcularLocaisNefastos(ED* ed, ED_LocaisNefastos* locaisNefastos, int max_linhas, int max_colunas) {
    if (ed == NULL || locaisNefastos == NULL) return; // Verificação de segurança para evitar erros

    Antena* atual = ed->cabeca;
    ED* mapaFrequencias[256] = { 0 }; // Array para armazenar listas de antenas por frequência (ASCII)

    // Passo 1: Agrupar antenas por frequência
    while (atual != NULL) {
        char freq = atual->frequencia;
        if (mapaFrequencias[freq] == NULL) {
            mapaFrequencias[freq] = criarED(); // Cria uma nova lista para armazenar antenas dessa frequência
        }
        insereAntena(mapaFrequencias[freq], freq, atual->linha, atual->coluna); // Insere a antena na lista correta
        atual = atual->prox;
    }

    // Passo 2: Calcular locais nefastos para cada par de antenas de mesma frequência
    for (int freq = 0; freq < 256; freq++) {
        if (mapaFrequencias[freq] == NULL) continue; // Se não houver antenas dessa frequência, pula para a próxima

        Antena* a1 = mapaFrequencias[freq]->cabeca;
        while (a1 != NULL) {
            Antena* a2 = a1->prox; // Evita pares duplicados (a1, a2) e (a2, a1)
            while (a2 != NULL) {
                // Diferença entre as coordenadas das duas antenas
                int dx = a2->coluna - a1->coluna;
                int dy = a2->linha - a1->linha;

                // Calcular as duas possíveis posições com efeito nefasto
                int L1_linha = a1->linha + 2 * dy;
                int L1_coluna = a1->coluna + 2 * dx;

                int L2_linha = a2->linha - 2 * dy;
                int L2_coluna = a2->coluna - 2 * dx;

                // Validação: Garante que os locais nefastos calculados estão dentro dos limites da matriz
                if (L1_linha >= 0 && L1_linha < max_linhas && L1_coluna >= 0 && L1_coluna < max_colunas) {
                    adicionarLocalNefasto(locaisNefastos, L1_linha, L1_coluna);
                }

                if (L2_linha >= 0 && L2_linha < max_linhas && L2_coluna >= 0 && L2_coluna < max_colunas) {
                    adicionarLocalNefasto(locaisNefastos, L2_linha, L2_coluna);
                }

                a2 = a2->prox; // Avança para o próximo par de antenas
            }
            a1 = a1->prox; // Avança para a próxima antena na lista
        }
    }

    // Passo 3: Liberar memória das listas temporárias usadas para agrupamento de antenas
    for (int i = 0; i < 256; i++) {
        if (mapaFrequencias[i] != NULL) {
            destruirED(mapaFrequencias[i]);
        }
    }
}

```

Figura 16 - Calcula Locais Nefastos

Listagem:

listarLocaisNefastos: Exibe coordenadas dos locais nefastos diretamente na consola. (Figura 17)

```

void listarLocaisNefastos(const ED_LocaisNefastos* locais) {
    // Verifica se a lista de locais nefastos está vazia
    if (locais == NULL || locais->cabeca == NULL) {
        printf("Nenhum local nefasto encontrado.\n");
        return;
    }

    LocalNefasto* atual = locais->cabeca; // Ponteiro para percorrer a lista ligada
    printf("=== Locais Nefastos ===\n");

    // Percorre a lista e imprime cada local nefasto
    while (atual != NULL) {
        printf("Local nefasto em (%d, %d)\n", atual->linha, atual->coluna);
        atual = atual->prox; // Avança para o próximo nó da lista
    }

    printf("=====\n");
}

```

Figura 17 - Listar Locais Nefastos

listarLocaisNefastosFormatado: Cria uma matriz visual com caracteres # para locais nefastos e "." para áreas livres. (Figura 18)

```

void listarLocaisNefastosFormatado(const ED_LocaisNefastos* locais, int max_linhas, int max_colunas) {
    // Verifica se a estrutura de locais é válida e se contém locais
    if (locais == NULL || locais->cabeca == NULL) {
        printf("Nenhum local nefasto encontrado.\n");
        return; // Se não houver locais nefastos, exibe mensagem e retorna
    }

    // Aloca dinamicamente a matriz de caracteres para armazenar a representação do mapa
    char** matriz = (char**)malloc(max_linhas * sizeof(char*)); // Aloca o vetor de ponteiros para as linhas
    for (int i = 0; i < max_linhas; i++) {
        matriz[i] = (char*)malloc(max_colunas * sizeof(char)); // Aloca cada linha da matriz
        for (int j = 0; j < max_colunas; j++) {
            matriz[i][j] = '.'; // Inicializa cada posição da matriz com o caractere '.'
        }
    }

    // Preenche a matriz com os locais nefastos representados por '#'
    LocalNefasto* atual = locais->cabeca; // Acessa o primeiro local da lista
    while (atual != NULL) {
        // Marca o local nefasto na posição correspondente da matriz
        matriz[atual->linha][atual->coluna] = '#';
        atual = atual->prox; // Avança para o próximo local na lista
    }

    // Imprime a matriz formatada, representando o mapa de locais nefastos
    printf("=== Mapa de Locais Nefastos ===\n");
    for (int i = 0; i < max_linhas; i++) {
        for (int j = 0; j < max_colunas; j++) {
            printf("%c", matriz[i][j]); // Imprime cada caractere da matriz
        }
        printf("\n"); // Quebra de linha ao final de cada linha da matriz
    }
    printf("=====\n");

    // Libera a memória alocada para a matriz
    for (int i = 0; i < max_linhas; i++) {
        free(matriz[i]); // Liberta cada linha da matriz
    }
    free(matriz); // Liberta o vetor de ponteiros para as linhas
}

```

Figura 18 - Lista Locais Nefastos Formatado

2.5. Projeto_EDA.cpp (main.c)

Por fim criou-se o arquivo principal (main) do projeto que conta com as seguintes funcionalidades:

2.5.1. **exibirMenu (Menu Interativo)**

Exibe opções para listar antenas, remover/inserir antenas, calcular e listar locais nefastos.

Utiliza um loop para manter o programa em execução até a escolha da opção de sair.

```
void exibirMenu() {  
    printf("\nMenu:\n");  
    printf("1 - Listar antenas\n");  
    printf("2 - Listar antenas formatadas\n");  
    printf("3 - Remover uma antena\n");  
    printf("4 - Inserir uma antena\n");  
    printf("5 - Calcular locais nefastos\n");  
    printf("6 - Listar locais nefastos\n");  
    printf("7 - Listar locais nefastos formatados\n");  
    printf("8 - Sair\n");  
    printf("Escolha uma opcao: ");  
}
```

Figura 19 - Menu Interativo

2.5.2. **main (Função principal).**

Gestão de Antenas:

Carrega as antenas a partir de um arquivo (Antenas.txt) no início.

Oferece opções para listar, inserir e remover antenas.

Usa funções específicas para operações com listas ligadas.

Cálculo e Listagem de Locais Nefastos:

Calcula os locais nefastos com base nas antenas cadastradas.

Oferece listagem simples e formatada dos locais nefastos.

Gestão de Memória:

Cria e destrói a estrutura de dados principal (ED) ao iniciar e encerrar o programa.

Liberta memória alocada ao sair.

```

int main() {
    int opcao;
    ED* ed = criarED();
    ED_LocaisNefastos locaisNefastos = { NULL };
    int max_linhas, max_colunas;

    // Carregar antenas do ficheiro antes de exibir o menu
    carregarAntenasDeFicheiro(ed, nomeArquivo, &max_linhas, &max_colunas);

    do {
        exibirMenu();
        scanf_s("%d", &opcao);

        switch (opcao) {
            case 1:
                listarAntenas(ed);
                break;
            case 2:
                listarAntenasFormatado(ed, max_linhas, max_colunas);
                break;
            case 3:
                listarAntenas(ed);
                removerAntenaManual(ed);
                break;
            case 4:
                listarAntenas(ed);
                inserirAntenaManual(ed);
                break;
            case 5:
                calcularLocaisNefastos(ed, &locaisNefastos, max_linhas, max_colunas);
                printf("Calculo feito com sucesso.\n");
                break;
            case 6:
                listarLocaisNefastos(&locaisNefastos);
                break;
            case 7:
                listarLocaisNefastosFormatado(&locaisNefastos, max_linhas, max_colunas);
                break;
            case 8:
                printf("A Sair...\n");
                break;
            default:
                printf("Opcao invalida! Tente novamente.\n");
        }
    } while (opcao != 8);

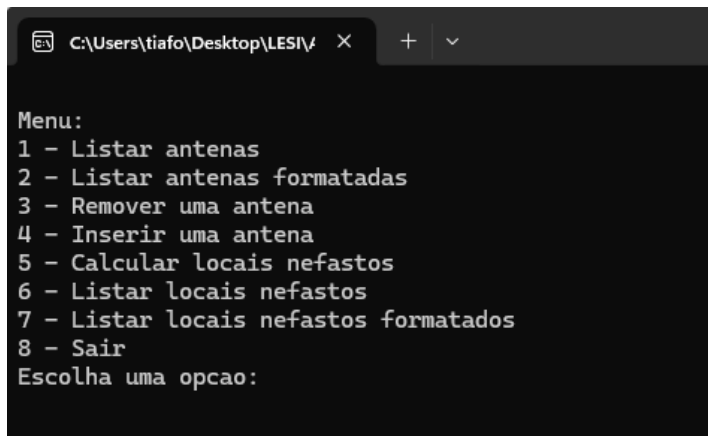
    destruirED(ed);
    return 0;
}

```

Figura 20 - Função principal (Main)

2.6. Testes ao programa.

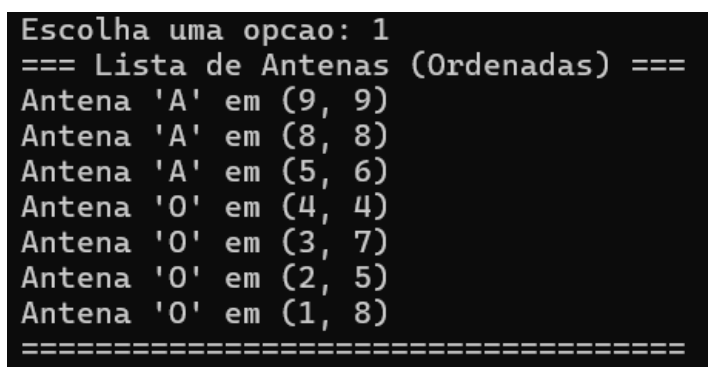
2.6.1. Menu



```
C:\Users\tiafo\Desktop\LESI\>
Menu:
1 - Listar antenas
2 - Listar antenas formatadas
3 - Remover uma antena
4 - Inserir uma antena
5 - Calcular locais nefastos
6 - Listar locais nefastos
7 - Listar locais nefastos formatados
8 - Sair
Escolha uma opcao:
```

Figura 21 - Menu

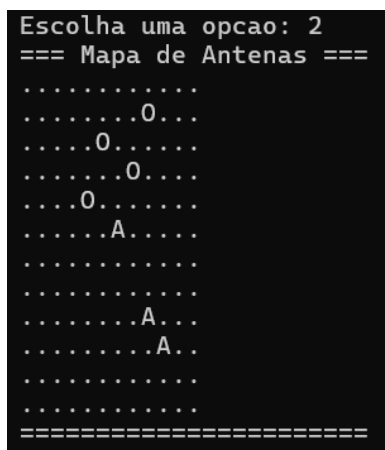
2.6.2. Listar Antenas



```
Escolha uma opcao: 1
=== Lista de Antenas (Ordenadas) ===
Antena 'A' em (9, 9)
Antena 'A' em (8, 8)
Antena 'A' em (5, 6)
Antena 'O' em (4, 4)
Antena 'O' em (3, 7)
Antena 'O' em (2, 5)
Antena 'O' em (1, 8)
=====
```

Figura 22 - Lista de Antenas Consola

2.6.3. Listar Antenas Formatado



```
Escolha uma opcao: 2
=== Mapa de Antenas ===
.....
.....0...
.....0...
.....0...
.....0...
.....A....
.....
.....
.....A...
.....A..
.....
.....
=====
```

Figura 23 - Lista Antenas Formatado Consola

2.6.4. Inserir Antena

```
Insira a frequencia da antena (caractere) a acrescentar: A
Insira a linha da antena: 5
Insira a coluna da antena: 5

Menu:
1 - Listar antenas
2 - Listar antenas formatadas
3 - Remover uma antena
4 - Inserir uma antena
5 - Calcular locais nefastos
6 - Listar locais nefastos
7 - Listar locais nefastos formatados
8 - Sair
Escolha uma opcao: 1
=== Lista de Antenas (Ordenadas) ===
Antena 'A' em (9, 9)
Antena 'A' em (8, 8)
Antena 'A' em (5, 5)
Antena 'A' em (5, 6)
Antena 'O' em (4, 4)
Antena 'O' em (3, 7)
Antena 'O' em (2, 5)
Antena 'O' em (1, 8)
=====
```

Figura 24 - Inserir Antena Consola

2.6.5. Remover Antena

```
=====
Insira a frequencia da antena (caractere) a remover: A
Insira a linha da antena: 5
Insira a coluna da antena: 5

Menu:
1 - Listar antenas
2 - Listar antenas formatadas
3 - Remover uma antena
4 - Inserir uma antena
5 - Calcular locais nefastos
6 - Listar locais nefastos
7 - Listar locais nefastos formatados
8 - Sair
Escolha uma opcao: 1
=== Lista de Antenas (Ordenadas) ===
Antena 'A' em (9, 9)
Antena 'A' em (8, 8)
Antena 'A' em (5, 6)
Antena 'O' em (4, 4)
Antena 'O' em (3, 7)
Antena 'O' em (2, 5)
Antena 'O' em (1, 8)
=====
```

Figura 25 - Remover Antena Consola

2.6.6. Listar Locais Nefastos

```
Escolha uma opcao: 6
=== Locais Nefastos ===
Local nefasto em (2, 10)
Local nefasto em (5, 1)
Local nefasto em (0, 6)
Local nefasto em (6, 3)
Local nefasto em (4, 9)
Local nefasto em (7, 0)
Local nefasto em (5, 6)
Local nefasto em (0, 11)
Local nefasto em (3, 2)
Local nefasto em (7, 7)
Local nefasto em (10, 10)
Local nefasto em (1, 3)
Local nefasto em (2, 4)
Local nefasto em (11, 10)
=====
```

Figura 26 - Lista Locais Nefastos Consola

2.6.7. Listar Locais Nefastos Formatado

```
=== Mapa de Locais Nefastos ===
.....#.....#
...#.....
....#.....#.
..#.....
.....#..
.#.....#.....
...#.....
#.....#.....
.....
.....
.....#
.....#
=====
```

Figura 27 - Lista Locais Nefastos Formatada Consola

3. Fase 2 – Grafos

Na segunda fase do projeto, avançou-se para a aplicação de conceitos avançados de teoria dos grafos para resolver o mesmo problema computacional relacionado às antenas e suas localizações com efeito nefasto. Enquanto a primeira fase utilizou listas ligadas para representar e manipular os dados das antenas, esta fase introduz uma abordagem mais sofisticada, modelando o problema como um grafo.

3.1. Atualização da estrutura de dados

Foram definidas estruturas de dados adicionais para representar o sistema de antenas como um grafo no ficheiro Dados.h. Estas novas estruturas complementam as existentes da Fase 1 e incluem:

Estruturas Principais:

Vertice:

Função: Representa um nó do grafo, associado a uma antena específica.

Campos:

- antena: Ponteiro para os dados da antena (frequência e coordenadas).
- adjacencias: Array de ponteiros para vértices vizinhos (antenas com mesma frequência).
- num_adjacencias: Contador do número de conexões do vértice.

GR (Grafo):

Função: Estrutura principal que armazena e faz a gestão todos os vértices do grafo.

Campos:

- vertices: Array de ponteiros para vértices.
- num_vertices: Número total de vértices no grafo.

O código referente a essa estrutura é o seguinte:

```

//FASE 2

/**
 * @brief Estrutura que representa um vértice no grafo
 */
typedef struct Vertice {
    Antena* antena;           // Dados da antena (frequência, linha, coluna)
    struct Vertice** adjacencias; // Lista de vértices adjacentes (mesma frequência)
    int num_adjacencias;      // Número de adjacências
} Vertice;

/**
 * @brief Estrutura que representa uma aresta no grafo
 */
typedef struct {
    Vertice** vertices;      // Array de vértices
    int num_vertices;        // Número total de vértices
} GR;

#endif // DADOS_H

```

Figura 28 - Estrutura de Dados da Fase 2

3.2. Grafo

3.2.1. Cabeçalho (Grafo.h)

Neste ficheiro de cabeçalho foram declaradas todas as funções desenvolvidas para gerir o grafo. Aqui foram declaradas as funções para:

- Criar um Grafo
- Destruir um Grafo
- Carregar um Grafo a partir da estrutura de Dados da Antena
- Inserir os Vértices no Grafo
- Inserir as Arestas num Grafo
- Listar os Vértices do Grafo
- Listar as Arestas do Grafo
- Procura em Profundidade

Pode verificar-se isso na figura seguinte:

```

#pragma once
#ifndef GRAFO_H
#define GRAFO_H

#include "Dados.h" // Para usar a estrutura Antena

// ----- Funções Básicas do Grafo -----
GR* criarGrafo();
void destruirGrafo(GR* grafo);
void carregarGrafoDeFicheiro(GR* grafo, ED* ed);
void inserirVertice(GR* grafo, Antena* antena);
void inserirAresta(GR* grafo, Antena* a1, Antena* a2);
void listarVertices(GR* grafo);
void listarArestas(GR* grafo);

// ----- Funções de Busca -----
void buscaProfundidade(GR* grafo, int linha, int coluna);

#endif // GRAFO_H

```

Figura 29 - Grafo.h

3.2.2. Implementação (Grafo.cpp)

Neste ficheiro foram implementadas todas as funções referidas no tópico anterior.

criarGrafo:

Função para criar uma estrutura de grafo vazia.

Aloca dinamicamente memória para a estrutura GR e inicializa os seus componentes básicos.

Em caso de falha na alocação de memória, exibe uma mensagem de erro e encerra o programa.

Retorna um ponteiro para o grafo criado. **(Figura 30)**

```

GR* criarGrafo() {
    // Aloca memória para a estrutura do grafo
    GR* grafo = (GR*)malloc(sizeof(GR));

    // Verifica se a alocação de memória foi bem-sucedida
    if (grafo == NULL) {
        perror("Erro ao alocar memória para o grafo"); // Exibe mensagem de erro
        exit(EXIT_FAILURE); // Encerra o programa com código de falha
    }

    // Inicializa a lista de vértices como NULL (grafo vazio)
    grafo->vertices = NULL;

    // Define o número inicial de vértices como zero
    grafo->num_vertices = 0;

    return grafo; // Retorna o ponteiro para o grafo recém-criado
}

```

Figura 30 – criar um Grafo

destruirGrafo:

Função para libertar toda a memória alocada para o grafo.

Percorre todos os vértices do grafo, libertando a memória de cada um deles e suas adjacências.

No final, liberta a memória da estrutura do grafo (**Figura 31**).

```

void destruirGrafo(GR* grafo) {
    for (int i = 0; i < grafo->num_vertices; i++) {
        free(grafo->vertices[i]->adjacencias);
        free(grafo->vertices[i]);
    }
    free(grafo->vertices);
    free(grafo);
}

```

Figura 31 - Destruir Grafo

carregarGrafo:

Função para carregar um grafo a partir de uma estrutura de dados de antenas (ED).

Percorre a lista ligada de antenas e insere cada antena como um vértice no grafo.

Em seguida, cria arestas entre todas as antenas que compartilham a mesma frequência de ressonância.(Figura 32)

```
void carregarGrafo(GR* grafo, ED* ed) {  
    // Percorre a lista ligada de antenas e adiciona cada uma como vértice no grafo  
    Antena* atual = ed->cabeca;  
    while (atual != NULL) {  
        inserirVertice(grafo, atual); // Insere a antena atual como vértice  
        atual = atual->prox; // Avança para a próxima antena na lista  
    }  
  
    atual = ed->cabeca; // Reinicia o ponteiro para o início da lista  
    while (atual != NULL) {  
        Antena* outra = atual->prox; // Ponteiro auxiliar para comparar com as antenas seguintes  
        while (outra != NULL) {  
            // Verifica se as antenas atual e 'outra' têm a mesma frequência  
            if (atual->frequencia == outra->frequencia) {  
                inserirAresta(grafo, atual, outra); // Cria aresta entre as antenas  
            }  
            outra = outra->prox; // Avança para a próxima antena na lista  
        }  
        atual = atual->prox; // Avança para a próxima antena principal  
    }  
}
```

Figura 32 - Carrega Grafo

inserirVertice:

Função para inserir um vértice (antena) no grafo.

Verifica se a antena já existe no grafo antes de inseri-la.

Se não existir, aloca memória para o novo vértice e o adiciona ao grafo.


```

void inserirVertice(GR* grafo, Antena* antena) {
    // Verifica se a antena já está no grafo
    for (int i = 0; i < grafo->num_vertices; i++) {
        if (grafo->vertices[i]->antena == antena) {
            return; // Antena já existe, não faz nada
        }
    }

    // Aloca memória para o novo vértice
    Vertice* novo = (Vertice*)malloc(sizeof(Vertice));
    if (novo == NULL) {
        perror("Erro ao alocar memória para o vértice");
        exit(EXIT_FAILURE);
    }

    // Inicializa os campos do novo vértice
    novo->antena = antena;
    novo->adjacencias = NULL;
    novo->num_adjacencias = 0;

    // Redimensiona o array de vértices do grafo
    Vertice** temp = (Vertice**)realloc(
        grafo->vertices,
        (grafo->num_vertices + 1) * sizeof(Vertice*)
    );

    if (temp == NULL) {
        free(novo); // Libera o vértice recém-criado
        perror("Erro ao realocar memória para os vértices");
        exit(EXIT_FAILURE);
    }

    // Atualiza o array de vértices e o contador
    grafo->vertices = temp;
    grafo->vertices[grafo->num_vertices] = novo;
    grafo->num_vertices++;
}

```

Figura 33 - Inserir Vértice

inserirAresta:

Função para inserir uma aresta entre dois vértices (antenas) no grafo.

Verifica se ambos os vértices existem e têm a mesma frequência antes de criar a aresta.

Adiciona cada vértice à lista de adjacências do outro (grafo não direcionado). **(Figura 34)**

```

void inserirAresta(GR* grafo, Antena* a1, Antena* a2) {

    // Se o grafo ou qualquer uma das antenas for nulo, a função termina sem fazer nada
    if (grafo == NULL || a1 == NULL || a2 == NULL) return;

    Vertice* v1 = NULL, * v2 = NULL;

    // Percorre todos os vértices do grafo para identificar os que correspondem às antenas a1 e a2
    for (int i = 0; i < grafo->num_vertices; i++) {
        if (grafo->vertices[i]->antena == a1) v1 = grafo->vertices[i];
        if (grafo->vertices[i]->antena == a2) v2 = grafo->vertices[i];
    }

    // A aresta só é inserida se ambos os vértices forem encontrados
    // e as antenas tiverem a mesma frequência
    if (v1 && v2 && v1->antena->frequencia == v2->antena->frequencia) {

        // Adiciona v2 à lista de adjacências de v1
        Vertice** nova_adj_v1 = (Vertice**)realloc(
            v1->adjacencias,
            (v1->num_adjacencias + 1) * sizeof(Vertice*)
        );

        // Verifica se a realocação de memória falhou
        if (nova_adj_v1 == NULL) {
            perror("Erro ao realocar memória para adjacências de v1");
            return;
        }

        // Atualiza a lista de adjacências de v1 e incrementa o número de adjacências
        v1->adjacencias = nova_adj_v1;
        v1->adjacencias[v1->num_adjacencias++] = v2;

        // Adiciona v1 à lista de adjacências de v2
        Vertice** nova_adj_v2 = (Vertice**)realloc(
            v2->adjacencias,
            (v2->num_adjacencias + 1) * sizeof(Vertice*)
        );

        // Verifica se a realocação de memória falhou
        if (nova_adj_v2 == NULL) {
            // Desfaz a alteração feita anteriormente em v1 para manter a consistência
            v1->adjacencias = (Vertice**)realloc(
                v1->adjacencias,
                (v1->num_adjacencias - 1) * sizeof(Vertice*)
            );
            v1->num_adjacencias--;

            perror("Erro ao realocar memória para adjacências de v2");
            return;
        }

        // Atualiza a lista de adjacências de v2 e incrementa o número de adjacências
        v2->adjacencias = nova_adj_v2;
        v2->adjacencias[v2->num_adjacencias++] = v1;
    }
}

```

Figura 34 - Insere Aresta

listarVertices:

Função para listar os vértices do grafo por ordem decrescente da linha e crescente da coluna.

A função cria um array temporário com os ponteiros para os vértices, ordena-os segundo os critérios especificados e imprime a informação de cada antena. **(Figura 35)**

```

void listarVertices(GR* grafo) {

    // Verifica se o grafo é válido e contém vértices
    if (grafo == NULL || grafo->num_vertices == 0) {
        printf("Nenhum vertex no grafo.\n");
        return;
    }

    // Copia os ponteiros dos vértices para um array auxiliar
    Vertice** arrayVertices = (Vertice**)malloc(grafo->num_vertices * sizeof(Vertice*));
    for (int i = 0; i < grafo->num_vertices; i++) {
        arrayVertices[i] = grafo->vertices[i];
    }

    // Ordena o array de vértices:
    // - Primeiro por linha (ordem decrescente)
    // - Em caso de empate, por coluna (ordem crescente)
    qsort(arrayVertices, grafo->num_vertices, sizeof(Vertice*),
    [](const void* a, const void* b) -> int {
        Vertice* vA = *(Vertice**)a;
        Vertice* vB = *(Vertice**)b;

        if (vA->antena->linha != vB->antena->linha) {
            return vB->antena->linha - vA->antena->linha;
        }

        return vA->antena->coluna - vB->antena->coluna;
    });

    // Imprime os vértices ordenados com as suas respectivas posições e frequências
    printf("=== Vertices do Grafo (Ordenados) ===\n");
    for (int i = 0; i < grafo->num_vertices; i++) {
        printf("Antena '%c' em (%d, %d)\n",
            arrayVertices[i]->antena->frequencia,
            arrayVertices[i]->antena->linha,
            arrayVertices[i]->antena->coluna);
    }
    printf("=====\n");

    // Liberta a memória alocada para o array auxiliar
    free(arrayVertices);
}

```

Figura 35 - Listar Vértices

listarArestas:

Função para listar as arestas do grafo.

Percorre todos os vértices e imprime as antenas às quais cada um está ligado.

(Figura 36)

```

void listarArestas(GR* grafo) {

    // Verifica se o grafo é válido
    if (grafo == NULL || grafo->num_vertices == 0) {
        printf("Grafo vazio ou inválido.\n");
        return;
    }

    // Percorre todos os vértices do grafo
    for (int i = 0; i < grafo->num_vertices; i++) {
        Vertice* v = grafo->vertices[i];

        // Imprime a antena do vértice atual
        printf("%c(%d, %d) está conectado a:\n",
            v->antena->frequencia,
            v->antena->linha,
            v->antena->coluna);

        // Percorre e imprime as antenas vizinhas (adjacentes)
        for (int j = 0; j < v->num_adjacencias; j++) {
            Vertice* vizinho = v->adjacencias[j];
            printf("  - %c(%d, %d)\n",
                vizinho->antena->frequencia,
                vizinho->antena->linha,
                vizinho->antena->coluna);
        }
    }
}

```

Figura 36 - Listar Arestas

auxProfundidade:

Função auxiliar recursiva para realizar a Procura por Profundidade. (Figura 37)

```

void auxProfundidade(Vertice* v, bool visitado[]) {

    // Marca o vértice atual como visitado
    // Utiliza uma hash simples com base nas coordenadas (linha * 1000 + coluna)
    visitado[v->antena->linha * 1000 + v->antena->coluna] = true;

    // Imprime a informação da antena visitada
    printf("Antena '%c' em (%d, %d)\n",
        v->antena->frequencia,
        v->antena->linha,
        v->antena->coluna);

    // Percorre os vizinhos do vértice
    for (int i = 0; i < v->num_adjacencias; i++) {
        Vertice* vizinho = v->adjacencias[i];

        // Calcula o índice hash do vizinho
        int hash = vizinho->antena->linha * 1000 + vizinho->antena->coluna;

        // Visita o vizinho se ainda não tiver sido visitado
        if (!visitado[hash]) {
            auxProfundidade(vizinho, visitado);
        }
    }
}

```

Figura 37 - Função auxiliar para procura de Profundidade

buscaProfundidade:

Função para iniciar a busca em profundidade (DFS) no grafo a partir de uma antena específica. (Figura 38)

```
void buscaProfundidade(GR* grafo, int linha, int coluna) {  
  
    // Verifica se o grafo é válido e contém vértices  
    if (grafo == NULL || grafo->num_vertices == 0) {  
        printf("Grafo vazio ou invalido.\n");  
        return;  
    }  
  
    // Procura o vértice correspondente às coordenadas fornecidas  
    Vertice* inicio = NULL;  
    for (int i = 0; i < grafo->num_vertices; i++) {  
        if (grafo->vertices[i]->antena->linha == linha &&  
            grafo->vertices[i]->antena->coluna == coluna) {  
            inicio = grafo->vertices[i];  
            break;  
        }  
    }  
  
    // Se não encontrar o vértice, informa o utilizador e termina  
    if (inicio == NULL) {  
        printf("Antena em (%d, %d) não encontrada.\n", linha, coluna);  
        return;  
    }  
  
    // Cria e inicializa o array de visitados, utilizando uma hash simples baseada nas coordenadas  
    // Assume que as coordenadas máximas são até 999x999  
    bool visitado[1000000] = { false };  
  
    // Inicia a procura a partir do vértice encontrado  
    auxProfundidade(inicio, visitado);  
}
```

Figura 38 - Função que procura em Profundidade

3.3. Projeto_EDA.cpp atualizado

Para a utilização e manipulação da nova estrutura de Dados do Grafo foi adicionada a diretiva **#include "Grafo.h"** para utilizar as funções de manipulação de grafos.

Foi criada a variável **GR* grafo** no main() para armazenar a estrutura do grafo e inicializada com **grafo = criarGrafo()**.

Para o carregamento do Grafo foi adicionada a chamada **carregarGrafo(grafo, ed)** após o carregamento das antenas do ficheiro, convertendo a lista ligada (ED) em estrutura de grafo.

3.3.1. Menu atualizado (exibirMenu)

Para a utilização das funções criadas o menu de interação foi atualizado com novas opções:

- ✓ Listar Vértices do Grafo
- ✓ Listar Arestas do Grafo
- ✓ Procura por profundidade

```
void exibirMenu() {  
    printf("\nMenu:\n");  
    printf("1 - Listar antenas\n");  
    printf("2 - Listar antenas formatadas\n");  
    printf("3 - Remover uma antena\n");  
    printf("4 - Inserir uma antena\n");  
    printf("5 - Calcular locais nefastos\n");  
    printf("6 - Listar locais nefastos\n");  
    printf("7 - Listar locais nefastos formatados\n");  
    printf("8 - Listar vertices do grafo\n");  
    printf("9 - Listar arestas do grafo\n");  
    printf("10 - Procura por profundidade\n");  
    printf("11 - Sair\n");  
    printf("Escolha uma opcao: ");  
}
```

Figura 39 - Menu atualizado

3.3.2. Main (função principal)

Na função principal foram adicionadas as funções relativas à escolha do utilizador como na seguinte figura. **(Figura 40).**

Utilizando a opção 10, o utilizador insere as coordenadas do Vértice que quer e esta faz a procura em profundidade dos Vértices interligados da mesma frequência.

```
case 8:  
    listarVertices(grafo);  
    break;  
case 9:  
    listarArestas(grafo);  
    break;  
case 10:  
    int linha, coluna;  
    printf("Digite as coordenadas da antena inicial (linha coluna): ");  
    scanf_s("%d %d", &linha, &coluna);  
    buscaProfundidade(grafo, linha, coluna);  
    break;  
case 11:  
    printf("A Sair...\n");  
    break;
```

Figura 40 - Novas funcionalidade do programa

3.4. Novos testes ao programa

3.4.1. Listagem de Vértices

```
Menu:
1 - Listar antenas
2 - Listar antenas formatadas
3 - Remover uma antena
4 - Inserir uma antena
5 - Calcular locais nefastos
6 - Listar locais nefastos
7 - Listar locais nefastos formatados
8 - Listar vertices do grafo
9 - Listar arestas do grafo
10 - Procura por profundidade
11 - Sair
Escolha uma opcao: 8
=== Vertices do Grafo (Ordenados) ===
Antena 'A' em (9, 9)
Antena 'A' em (8, 8)
Antena 'A' em (5, 6)
Antena 'O' em (4, 4)
Antena 'O' em (3, 7)
Antena 'O' em (2, 5)
Antena 'O' em (1, 8)
=====
```

Figura 41 - Listar Vertices

3.4.2. Listagem de Arestas

```
Escolha uma opcao: 9
A(9, 9) esta conectado a:
- A(8, 8)
- A(5, 6)
A(8, 8) esta conectado a:
- A(9, 9)
- A(5, 6)
A(5, 6) esta conectado a:
- A(9, 9)
- A(8, 8)
O(4, 4) esta conectado a:
- O(3, 7)
- O(2, 5)
- O(1, 8)
O(3, 7) esta conectado a:
- O(4, 4)
- O(2, 5)
- O(1, 8)
O(2, 5) esta conectado a:
- O(4, 4)
- O(3, 7)
- O(1, 8)
O(1, 8) esta conectado a:
- O(4, 4)
- O(3, 7)
- O(2, 5)
```

Figura 42 - Listagem de Arestas

3.4.3. Procura por Profundidade

```
Escolha uma opcao: 10
Digite as coordenadas da antena inicial (linha coluna): 4 4
Antena '0' em (4, 4)
Antena '0' em (3, 7)
Antena '0' em (2, 5)
Antena '0' em (1, 8)
```

Figura 43 - Procura por Profundidade

4. Conclusão

A implementação deste projeto permitiu a consolidação e aplicação prática de conceitos fundamentais e avançados de estruturas de dados, ao longo de duas fases distintas, complementares entre si.

Na primeira fase, foi feita a modelação do problema através da utilização de listas ligadas para representar antenas e localizações com efeito nefasto. As funcionalidades desenvolvidas incluíram a leitura de matrizes a partir de ficheiros, a criação e manipulação dinâmica da lista de antenas, bem como a dedução automática dos locais nefastos com base nas regras de alinhamento e distância entre antenas com a mesma frequência. A implementação revelou-se robusta e modular, com especial atenção à gestão eficiente da memória e à apresentação clara da informação ao utilizador, através de um menu interativo.

Na segunda fase, o projeto evoluiu com a introdução de estruturas de grafos, possibilitando uma análise mais avançada das relações entre antenas. As antenas foram representadas como vértices e as arestas criadas com base na partilha da mesma frequência. Foram implementadas funcionalidades de procura em profundidade (DFS) permitindo explorar conexões a partir de uma antena específica. Esta fase evidenciou uma compreensão sólida dos conceitos de grafos e a sua aplicação eficaz em problemas do mundo real.

Globalmente, o projeto demonstrou um progresso consistente na complexidade e profundidade das soluções desenvolvidas, com uma transição natural entre diferentes paradigmas de modelação de dados. A integração de técnicas de programação estruturada, e a realização de testes reforçou a qualidade e a escalabilidade da solução final. Este trabalho representou uma oportunidade valiosa de aprendizagem, tanto a nível técnico como metodológico, contribuindo significativamente para o desenvolvimento de competências em programação avançada e resolução de problemas computacionais.

5. Bibliografia

Link Repositório GIT : https://github.com/Tiago20452/Projeto_EDA_20452.git

Código desenvolvido pelo professor.

<https://www.ime.usp.br/~pf/algoritmos/aulas/lista.html>

<https://seguranca-informatica.pt/lista-ligada-c/>

[https://www.ime.usp.br/~pf/algoritmos_para_grafos/aulas/graphdatastructs.ht](https://www.ime.usp.br/~pf/algoritmos_para_grafos/aulas/graphdatastructs.html)

[ml](#)

https://www.ime.usp.br/~pf/algoritmos_para_grafos/